Matroid Theory

from First Principles & Its Role in Greedy Algorithms

A rigorous yet accessible introduction
for advanced undergraduates and early graduate students

Topics: Matroid Axioms · Greedy Theorem · Graphic Matroids · MST Algorithms

Part I

Motivation

Why do some greedy algorithms work — and others fail?

The Central Question: When Does Greedy Work?

When does greedily choosing the locally best option yield a globally optimal solution?

- Greedy algorithms are simple, fast, and elegant.
- But they are *not always correct*.
- **Matroids** reveal exactly when and why greedy is provably optimal.

Answer: The greedy algorithm finds a maximum-weight solution for *every* weight function **if and only if** the feasible sets form a matroid.

A Greedy Success: Select the k Largest Items

Problem: Choose at most k items from a set to maximise total value.

Example: Greedy Works!
Items with values $\{9, 5, 3, 1\}$, $k = 2$.
Greedy: pick 9, then 5. Total $= 14$. This is optimal.

Why does greedy work here? • Any subset of size $\leq k$ is feasible.
• If our set has fewer than k items and a better unused item exists, we can freely add it.
• No feasibility constraint ever *blocks* a greedy choice.
  – This is the **uniform matroid** in disguise — we will formalise it shortly.

A Greedy Failure: 0/1 Knapsack

Problem: Capacity $= 10$. Greedy by value/weight ratio:

Item	Weight	Value	Val/Wt
A	6	12	2.0
B	5	9	1.8
C	5	9	1.8

Non-Example: Greedy by ratio fails!
Greedy picks A (weight 6), then cannot fit B or C. Total value $= 12$.
Optimal: $B + C$ (weight $= 10$, value $= 18$).

- The feasible sets $\{S : \text{weight}(S) \leq 10\}$ do **not** form a matroid.
- The **exchange axiom** fails — this is why greedy breaks.

Independence: The Common Abstraction

Both examples involve a collection of *feasible subsets*. The key is their structure:

Setting	Ground Set E	Independent Sets I
Choose $\leq k$ items	Items	Subsets of size $\leq k$
Linear algebra	Vectors	Linearly independent subsets
Graph theory	Edges	Acyclic edge sets (forests)
Scheduling	Jobs	Feasible job subsets

A **matroid** $M = (E, \mathcal{I})$ abstracts precisely the structure shared by all of these — enabling a single unified proof of greedy correctness.

Part II

First Principles

Defining matroids from axioms

Definition of a Matroid

Definition: Matroid $M = (E, \mathcal{I})$

A **matroid** is a pair $M = (E, \mathcal{I})$ where E is a finite set and $\mathcal{I} \subseteq 2^E$ satisfies:

(I1) Non-emptiness: $\emptyset \in \mathcal{I}$
(I2) Heredity: If $A \in \mathcal{I}$ and $B \subseteq A$, then $B \in \mathcal{I}$
(I3) Exchange: If $A, B \in \mathcal{I}$ and $|A| < |B|$, then $\exists x \in B \setminus A$ such that $A \cup \{x\} \in \mathcal{I}$

- Elements of $\mathcal{I}$ are called **independent**; others are **dependent**.
- E is the **ground set**. Examples: columns of a matrix, edges of a graph, items in a set.

Axiom	Name	Informal Meaning
(I1)	Non-emptiness	Choosing nothing is always feasible
(I2)	Heredity	Subsets of feasible sets are feasible
(I3)	Exchange	Smaller independent sets can be augmented

Axiom (I1): Non-Emptiness

(I1) The empty set is independent: $\emptyset \in \mathcal{I}$

Interpretation: • Choosing nothing is always feasible — a base case for building independent sets.
• Without (I1), we could not guarantee any solution exists.

In each setting: • **Vectors:** The empty set of vectors is trivially linearly independent.
• **Edges:** The empty edge set is a trivially acyclic forest.
• **Selection:** Choosing zero items satisfies any cardinality bound.

Axiom (I2): Heredity (Downward Closure)

(I2) If $A \in \mathcal{I}$ and $B \subseteq A$, then $B \in \mathcal{I}$

Interpretation: Subsets of independent sets are independent. Independence is *downward-closed.*

In each setting: • **Vectors:** Any subset of linearly independent vectors is linearly independent.
• **Edges:** Any subset of an acyclic edge set is also acyclic.
• **Selection:** If choosing k items is feasible, choosing fewer is also feasible.

Non-Example: The Knapsack feasible system violates heredity!
Feasible sets are $\{S : \text{weight}(S) \leq 10\}$. Suppose $\{B, C\}$ is feasible but $\{A, B, C\}$ is not. Yet individually A has weight $6 \leq 10$. The issue: adding A to $\{B, C\}$ *exceeds* capacity, so $\{A, B, C\} \notin \mathcal{I}$. The downward-closure property *does* hold here individually, but exchange fails.

Axiom (I3): Exchange (Augmentation) — The Key Axiom

(I3) If $A, B \in \mathcal{I}$ and $|A| < |B|$, then $\exists x \in B \setminus A$ such that $A \cup \{x\} \in \mathcal{I}$

Interpretation: Any smaller independent set can be *augmented* from a larger one. • We can always "fill up" a small independent set using elements of a bigger one.

• This is the **heart of greedy correctness** — we prove this formally in Part IV.

• In linear algebra: if $\dim(A) < \dim(B)$, we can find a vector in B not in $\text{span}(A)$.

Why this matters algorithmically:
- Exchange prevents greedy from reaching a "dead end" — a locally maximal but globally sub-optimal solution.
- It implies all maximal independent sets (bases) have equal size — no "accidentally small" solutions.

Uniform Matroids $U_{k,n}$

Definition: Uniform Matroid $U_{k,n}$
Ground set: $E = \{1, 2, \ldots, n\}$.
Independent sets: $\mathcal{I} = \{A \subseteq E : |A| \leq k\}$.

Example: $U_{2,4}$ — $E = \{a, b, c, d\}$, $k = 2$
$\mathcal{I} = \{\emptyset, \{a\}, \{b\}, \{c\}, \{d\}, \{a, b\}, \{a, c\}, \{a, d\}, \{b, c\}, \{b, d\}, \{c, d\}\}$
Dependent sets: $\{a, b, c\}, \{a, b, d\}, \{a, c, d\}, \{b, c, d\}, \{a, b, c, d\}$

Axiom verification: • (I1) $\emptyset \in \mathcal{I}$ since $|\emptyset| = 0 \leq k$. ✓
• (I2) If $|A| \leq k$ and $B \subseteq A$, then $|B| \leq |A| \leq k$. ✓
• (I3) If $|A| < |B| \leq k$, pick any $x \in B \setminus A$. Then $|A \cup \{x\}| = |A| + 1 \leq |B| \leq k$. ✓

• **Greedy on $U_{\{k,n\}}$:** simply select the k highest-weight elements. Trivially optimal.

Linear Matroids

Definition: Linear Matroid

Let $\mathbb{F}$ be a field, A an $m \times n$ matrix over $\mathbb{F}$.
Ground set: $E = \{1, \ldots, n\}$ (column indices).
Independent sets: $\mathcal{I} = \{S \subseteq E : \text{columns indexed by } S \text{ are linearly independent}\}$.

Example: Over $\mathbb{R}$

$A = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix}$. Columns: $v_1 = (1,0)$, $v_2 = (0,1)$, $v_3 = (1,1)$.
Independent: $\{1\}, \{2\}, \{3\}, \{1,2\}, \{1,3\}, \{2,3\}$.
Dependent: $\{1,2,3\}$ since $v_3 = v_1 + v_2$.

- Exchange axiom $\leftrightarrow$ the linear-algebra augmentation lemma for bases.
- Rank of matroid = column rank of A.
- Used in: network coding, VLSI, maximum-weight closure, matching in bipartite graphs.

Graphic Matroids $M(G)$

Definition: Graphic Matroid $M(G)$
Let $G = (V, E)$ be an undirected graph.
Ground set: E (edges of G).
Independent sets: $\mathcal{J} = \{F \subseteq E : (V, F) \text{ is a forest (acyclic)}\}$.

Structure: • Bases = spanning forests; in connected G: bases = spanning trees (each with $|V| - 1$ edges).
• Circuits = simple cycles of G.
• Rank: $r(F) = |V| - (\text{number of connected components of } (V, F))$.

Axiom verification: • (I1) Empty edge set is an acyclic forest. ✓
• (I2) Any subset of an acyclic edge set is acyclic. ✓
• (I3) Two forests F_1, F_2 with $|F_1| < |F_2|$: F_2 has fewer components, so some edge of F_2 connects two components of F_1 — adding it stays acyclic. ✓

A Non-Example: Failure of the Exchange Axiom

Non-Example: Non-matroid: mismatched maximal sets

$E = \{a, b, c\}$, $\mathcal{I} = \{\emptyset, \{a\}, \{b, c\}\}$.

Both $\{a\}$ and $\{b, c\}$ are maximal (cannot add any element).

But $|\{a\}| = 1 \neq 2 = |\{b, c\}|$.

Exchange fails: from $\{b, c\}$, we cannot augment $\{a\} - \{a, b\} \notin \mathcal{I}$ and $\{a, c\} \notin \mathcal{I}$.

Theorem: Diagnostic

If (I3) fails, there exist weights making greedy suboptimal.

Proof: Assign $w(a) = 5$, $w(b) = w(c) = 3$.

Greedy picks $\{a\}$ (value 5). But $\{b, c\}$ has value 6. Greedy fails.

Key lesson: Always verify all three matroid axioms before trusting a greedy algorithm. Failing (I3) is both necessary and sufficient for greedy to break on some weight function.

Part III

Structure

Bases · Rank · Circuits · Closure

Bases

Definition: Basis

A **basis** of $M = (E, \mathcal{I})$ is a maximal independent set: $B \in \mathcal{I}$ such that $B \cup \{e\} \notin \mathcal{I}$ for all $e \in E \setminus B$.
The collection of all bases is denoted $\mathcal{B}(M)$.

Theorem: Equal Cardinality of Bases

All bases of a matroid have the same cardinality.

Proof sketch: Suppose $|B_1| < |B_2|$. By (I3), $\exists e \in B_2 \setminus B_1$ with $B_1 \cup \{e\} \in \mathcal{I}$, contradicting maximality of B_1. ∎

Examples: • $U_{\{k,n\}}$: all bases have size exactly k.
• Linear matroid on A: bases are maximal linearly independent column sets — all have size $= \operatorname{rank}(A)$.
• Graphic matroid on connected G: bases = spanning trees, all of size $|V| - 1$.

Rank Function

Definition: Rank $r : 2^E \to \mathbb{Z}_{\geq 0}$

$$r(A) = \max\{|I| : I \in \mathcal{I}, I \subseteq A\}$$

For $A \subseteq E$.

The **rank of the matroid** is $r(E)$.

Key properties:
- (R1) $0 \leq r(A) \leq |A|$
- (R2) Monotone: $A \subseteq B \Rightarrow r(A) \leq r(B)$
- (R3) **Submodular**: $r(A \cup B) + r(A \cap B) \leq r(A) + r(B)$

In specific matroids:
- $U_{\{k,n\}}$: $r(A) = \min(|A|, k)$.
- Linear matroid: $r(A)$ = column rank of the submatrix with columns in A.
- Graphic matroid: $r(F) = |V| - c(F)$ where $c(F)$ = number of connected components.

Circuits

Definition: Circuit

A **circuit** is a minimal dependent set: $C \subseteq E$ with $C \notin \mathcal{I}$ but $C \setminus \{e\} \in \mathcal{I}$ for every $e \in C$.
The collection of all circuits is $\mathcal{C}(M)$.

In graphic matroids: • Circuits = simple cycles of G.
• Adding any edge to a spanning tree creates exactly one circuit.
• Cycle property: the heaviest edge in any cycle belongs to no MST.

Example:

Path $1\{-\}2\{-\}3\{-\}1$ is a circuit. Removing any edge yields an acyclic (independent) set.

Circuit properties: • No circuit is a subset of another: $\mathcal{C}(M)$ is an antichain.
• **Circuit elimination:** If $C_1, C_2 \in \mathcal{C}(M)$ and $e \in C_1 \cap C_2$, then $\exists C_3 \in \mathcal{C}(M)$ with $C_3 \subseteq (C_1 \cup C_2) \setminus \{e\}$.
• Matroids can be axiomatised via circuits equivalently.

Closure (Span)

Definition: Closure $\mathrm{cl} : 2^E \to 2^E$
$\mathrm{cl}(A) = \{e \in E : r(A \cup \{e\}) = r(A)\}$.
Equivalently: $e \in \mathrm{cl}(A)$ iff adding e to A does not increase rank.

Intuition: • $\mathrm{cl}(A)$ is the "span" of A — all elements already "dependent" on A.
• Linear matroids: $\mathrm{cl}(A) = \{\text{vectors in } E \text{ lying in } \mathrm{span}(A)\}$.
• Graphic matroids: $\mathrm{cl}(F) = F \cup \{\text{edges whose endpoints are connected in } (V, F)\}$.

Closure axioms (alternative axiom system for matroids): • (CL1) $A \subseteq \mathrm{cl}(A)$
• (CL2) $A \subseteq B \Rightarrow \mathrm{cl}(A) \subseteq \mathrm{cl}(B)$
• (CL3) $\mathrm{cl}(\mathrm{cl}(A)) = \mathrm{cl}(A)$
• (CL4) **Mac Lane–Steinitz:** $e \notin \mathrm{cl}(A)$ and $e \in \mathrm{cl}(A \cup \{f\}) \Rightarrow f \in \mathrm{cl}(A \cup \{e\})$

Part IV

Greedy Algorithms & Matroids

The central theorem and its proof

The Generic Greedy Algorithm for Matroids

```
GREEDY(M = (E, I), w : E ⟶ ℝ):
Sort elements: e₁, e₂, ..., eₙ with w(e₁) ≥ w(e₂) ≥ ... ≥ w(eₙ)
S ⟵ ∅
for i = 1 to n do:
if S ∪ {eᵢ} ∈ I then
S ⟵ S ∪ {eᵢ}
return S
```

Properties: • Time: $O(n \log n)$ for sorting + n independence oracle calls.

• Correctness: holds **if and only if** $(E, \mathcal{I})$ is a matroid.

• Independence oracle examples:

- $U_{\{k,n\}}$: check $|S| < k$. $O(1)$.
- Graphic: check cycle via Union-Find. $O(\alpha(n))$ amortised.
- Linear: check rank via Gaussian elimination. $O(n^2)$.

The Matroid Greedy Theorem

Theorem: Matroid Greedy Theorem (Edmonds, 1971)
Let $(E, \mathcal{I})$ be an independence system (satisfying I1 and I2). The greedy algorithm finds a maximum-weight basis for **every** weight function $w : E \to \mathbb{R}_{\{\geq 0\}}$ **if and only if** $(E, \mathcal{I})$ is a matroid.

Proof sketch ($\Rightarrow$) — matroids make greedy optimal: • Suppose greedy output S is suboptimal; let OPT be a better solution.
• Find the first weight-rank position where they diverge: greedy chose s_i, OPT has o_i with $w(o_i) \geq w(s_i)$.
• At that step, greedy had partial set S'. It rejected o_i, so $S' \cup \{o_i\} \notin \mathcal{I}$.
• But $|S'| < |\mathrm{OPT}_{\{(\leq i)\}}|$ and both are independent — by (I3), greedy should have been able to add some element from OPT. Contradiction. ∎

Proof sketch ($\Leftarrow$) — failing (I3) breaks greedy: • Construct weights that force greedy to pick a smaller maximal set over a larger one — demonstrating sub-optimality.

Exchange Axiom: Why Greedy Cannot Get Stuck

The exchange axiom is precisely what prevents greedy from reaching a "local maximum trap".

Without exchange: • Two maximal independent sets can have different sizes.
• Greedy might fill up a small maximal set, missing a larger (better-weight) one.
• No way to "repair" the partial solution.

With exchange: • All maximal independent sets have equal size (= rank of matroid).
• Greedy's partial solution is always extendable — it never gets permanently stuck.
• Any element missed by greedy could have been included (at the appropriate weight rank), contradicting greedy's sorted order.

Theorem:
A family $\mathcal{J}$ satisfying (I1) and (I2) is a matroid $\Leftrightarrow$ all maximal independent sets have equal cardinality (for every restriction to a subset $A \subseteq E$).

Part V

Greedy in Action

Specialising to classical algorithms

Partition Matroid: Best From Each Category

Definition: Partition Matroid

E partitioned into disjoint classes $E_1, E_2, \ldots, E_m$ with quotas $k_1, \ldots, k_m$.
$\mathcal{I} = \{S \subseteq E : |S \cap E_i| \leq k_i \quad \text{for all} \quad i\}$.

Example: Job scheduling by type

$E = \{\text{coding jobs}\} \cup \{\text{design jobs}\} \cup \{\text{testing jobs}\}$, quotas $k_1 = 2, k_2 = 1, k_3 = 2$.
Greedy: sort all jobs by value; pick each job if its category quota is not exceeded.

Greedy specialisation: • Sort all elements by weight descending.
• Add element $e \in E_i$ if $|S \cap E_i| < k_i$.
• This is optimal by the matroid greedy theorem.

Application: Bipartite matching = common independent set of two partition matroids (matroid intersection).

Graphic Matroid: Maximum-Weight Forest

Matroid: $M(G) = (E, \mathcal{I})$ where $E =$ edges, $\mathcal{I} =$ acyclic subsets.

Generic greedy on $M(G)$: • Sort edges by weight descending (for max forest) or ascending (for MST).
• Add each edge if it does not create a cycle (independence check via Union-Find).
• Output: a maximum-weight spanning forest.

Example: Small worked example
G: vertices $\{1, 2, 3, 4\}$, edges with weights: $e_{\{12\}} = 5, e_{\{23\}} = 3, e_{\{34\}} = 4, e_{\{14\}} = 2, e_{\{13\}} = 6$. Sorted desc: $e_{\{13\}} = 6, e_{\{12\}} = 5, e_{\{34\}} = 4, e_{\{23\}} = 3, e_{\{14\}} = 2$.
Step 1: Add $e_{\{13\}}$. Step 2: Add $e_{\{12\}}$. Step 3: Add $e_{\{34\}}$.
Step 4: $e_{\{23\}}$ creates cycle $1\{-\}2\{-\}3\{-\}1$ — reject. Step 5: $e_{\{14\}}$ creates cycle — reject.
Max forest: $\{e_{\{13\}}, e_{\{12\}}, e_{\{34\}}\}$, weight $= 15$.

For MST: negate weights (or sort ascending). Kruskal's algorithm is exactly this greedy!

Part VI

Minimum Spanning Trees

Kruskal · Prim · Cut and Cycle Properties

The Minimum Spanning Tree Problem

Definition: Minimum Spanning Tree
Given: connected undirected graph $G = (V, E)$, weight $w : E \to \mathbb{R}$.
Find: a spanning tree $T \subseteq E$ minimising $\sum_{e \in T} w(e)$.

Matroid connection: • Graphic matroid $M(G) = (E, \mathcal{I})$: $E =$ edges, $\mathcal{I} =$ forests.
• Bases = spanning trees. MST = minimum-weight basis of $M(G)$.
• By the matroid greedy theorem: processing edges in ascending weight order and adding cycle-free edges is optimal.

Applications: • Network design: cable routing, road construction, pipeline layout.
• Clustering: single-linkage hierarchical clustering.
• Approximation algorithms: MST gives a 2-approximation for TSP.
• Phylogenetics, image segmentation, circuit layout.

Kruskal's Algorithm

```
KRUSKAL(G = (V, E), w : E ⟶ ℝ):
Sort edges: e₁, e₂, ..., eₘ with w(e₁) ≤ w(e₂) ≤ ... ≤ w(eₘ)
T ⟵ ∅ DSU.init(V) for i = 1 to m do:
let eᵢ = (u, v)
if DSU.find(u) ≠ DSU.find(v) then T ⟵ T ∪ {eᵢ}
DSU.union(u, v)
return T
```

Analysis: • Independence check: $e_i = (u, v)$ is independent $\Leftrightarrow$ u and v are in different components of (V, T). DSU.find: $O(\alpha(|V|))$.
• Total time: $O(|E| \log |E|)$ dominated by sorting.
• Correctness: direct corollary of the matroid greedy theorem applied to $M(G)$.

Union-Find (Disjoint Set Union)

Purpose: Maintain a partition of V into connected components of the growing forest.

```
DSU.init(V): each vertex forms its own singleton component
DSU.find(u): return root/representative of the component containing u
DSU.union(u,v): merge the components of u and v

Key invariant: find(u) == find(v) ⟺ u and v are connected in current T
```

Optimisations: • **Union by rank:** always attach the shorter tree under the taller — keeps trees shallow.
• **Path compression:** during find, flatten all nodes directly to root — future finds faster.
• With both: amortised $O(\alpha(n))$ per operation, where α is the inverse Ackermann function.
• For all practical purposes: $\alpha(n) \leq 4$ for $n \leq 10^{\{80\}}$.

In Kruskal's: $|E|$ find operations + $|V| - 1$ union operations $\Rightarrow$ total DSU cost $\approx O(|E|)$.

Kruskal's Algorithm: Worked Example

Graph G: 5 vertices $\{A, B, C, D, E\}$, 7 edges.

Edge	Weight	Action
C–E	2	Add ✓
A–B	4	Add ✓
D–E	6	Add ✓
B–E	7	Add ✓
A–D	8	Skip (cycle)
B–C	8	Skip (cycle)
B–D	11	Skip (cycle)

MST found: $T = \{C\{-\}E, A\{-\}B, D\{-\}E, B\{-\}E\}$

Total weight $= 2 + 4 + 6 + 7 = \mathbf{19}$

After adding $C\{-\}E$: components $\{C, E\}, \{A\}, \{B\}, \{D\}$.
After $A\{-\}B$: $\{A, B\}, \{C, E\}, \{D\}$.
After $D\{-\}E$: $\{C, D, E\}, \{A, B\}$.
After $B\{-\}E$: $\{A, B, C, D, E\}$ — done!

Prim's Algorithm

```
PRIM(G = (V, E), w : E ⟶ ℝ, start s ∈ V):
key[v] ⟵ ∞ for all v ∈ V; key[s] ⟵ 0
parent[v] ⟵ NIL for all v ∈ V
Q ⟵ min-priority queue containing all vertices, keyed by key[·]
while Q ≠ ∅ do:
u ⟵ EXTRACT-MIN(Q)
for each neighbour v of u do:
if v ∈ Q and w(u,v) < key[v] then
parent[v] ⟵ u
key[v] ⟵ w(u,v) ▷ DECREASE-KEY in Q
return { (parent[v], v) : v ≠ s }
```

Analysis: • Time: $O(|E| \log |V|)$ with binary heap; $O(|E| + |V| \log |V|)$ with Fibonacci heap.

• Prim grows a **single tree** from s, always adding the cheapest edge to a new vertex.

• Correctness: justified by the **cut property** — the minimum-weight crossing edge of any cut is in some MST.

Cut Property and Cycle Property

Theorem: Cut Property
For any cut $(S, V \setminus S)$ of G, the minimum-weight edge crossing the cut belongs to some MST. If this edge is unique, it belongs to every MST.

Theorem: Cycle Property
For any cycle C in G, the maximum-weight edge of C does not belong to any MST. If this edge is unique, it belongs to no MST.

Connection to matroid structure: • Cut property $\leftrightarrow$ exchange axiom: a sub-optimal tree can be augmented.
• Cycle property $\leftrightarrow$ circuit elimination: the heaviest circuit edge is excluded.

Algorithmic use: • **Kruskal** uses the cycle property: skip edge if it creates a cycle.
• **Prim** uses the cut property: always take the cheapest edge out of the current tree.

Kruskal vs Prim: Comparison

Aspect	Kruskal's	Prim's
Strategy	Global edge sort	Local vertex expansion
Data structure	Union-Find (DSU)	Priority queue
Grows	A forest (multiple trees)	One tree from start node
Time	$O(\|E\| \log \|E\|)$	$O(\|E\| \log \|V\|)$ or better
Best for	Sparse graphs	Dense graphs (Fibonacci heap)
Matroid view	Direct graphic matroid greedy	Implicit; cut-property driven
Correctness basis	Matroid greedy theorem	Cut property

Both produce a minimum spanning tree. Kruskal aligns *directly* with the graphic matroid greedy algorithm; Prim's matroid interpretation is more implicit.

Part VII

Beyond MSTs

Matroid intersection · Submodularity · Generalisations

Matroid Intersection

Definition: Matroid Intersection

Given two matroids $M_1 = (E, \mathcal{I}_1)$ and $M_2 = (E, \mathcal{I}_2)$ on the same ground set, find a maximum-weight set $S \in \mathcal{I}_1 \cap \mathcal{I}_2$.

Example: Bipartite Matching

$G = (U \cup V, E)$. A matching = edge set where each vertex appears ≤ 1 time.
M_1: partition matroid on U — at most one edge per $u \in U$.
M_2: partition matroid on V — at most one edge per $v \in V$.
Maximum matching = max common independent set of M_1 and M_2.

Complexity: • Single matroid: greedy, $O(|E| \log |E|)$.
• Matroid intersection: solvable in polytime via augmenting paths, $O\left(|E|^{\{1.5\}} \cdot T_{\{\text{oracle}\}}\right)$.
• Three-matroid intersection: NP-hard in general!

Submodular Optimisation over Matroids

Definition: Submodular Function
$f : 2^E \to \mathbb{R}$ is **submodular** if for all $A, B \subseteq E$:
$f(A \cup B) + f(A \cap B) \leq f(A) + f(B)$
(equivalently: f has *diminishing marginal returns*).

Connection to matroids: • The rank function r of any matroid is submodular.
• Maximising a monotone submodular function subject to a matroid constraint: greedy achieves a $\left(1 - \frac{1}{e}\right) \approx 0.632$ approximation!

Applications: • **Influence maximisation** in social networks (viral marketing).
• **Feature selection** in machine learning.
• **Sensor placement** for maximum coverage.
• **Welfare maximisation** in combinatorial auctions.

Generalisations: Greedoids and Antimatroids

What if we relax matroid axioms?

Definition: Greedoid
$(E, \mathcal{F})$ with $\emptyset \in \mathcal{F}$ and: for $A, B \in \mathcal{F}$ with $|A| > |B|$, $\exists e \in A$ s.t. $B \cup \{e\} \in \mathcal{F}$.
Heredity (I2) is **not required**.

Examples of greedoids: • Branching greedoids (rooted spanning trees explored in BFS order).
• Ear decompositions of graphs.
• Gaussian elimination sequences.

Definition: Antimatroid
Satisfies heredity (I2) but not exchange. Models "reachable" sets: start from a feasible set and expand.
Examples: convex point sets, node-search in trees.

Greedoids and antimatroids extend matroid theory to capture even more greedy-like algorithms beyond the classical matroid framework.

Applications of Matroid Theory

Application Domain	Matroid concept used
Network design (cable, roads)	Graphic matroid — MST
Bipartite matching	Matroid intersection
Job scheduling	Partition / transversal matroids
Error-correcting codes	Linear matroids over finite fields
VLSI circuit layout	Linear matroids, connectivity
Influence / viral marketing	Submodular optimisation over matroids
Approximation algorithms (TSP)	MST as 2-approximation
Phylogenetics	MST for tree reconstruction
Image segmentation	Graph-cut / matroid methods
Combinatorial auctions	Submodular welfare maximisation

Part VIII

Summary & Takeaways

The big picture

Part VIII — Conclusion

Key Takeaways

- A **matroid** $M = (E, \mathcal{I})$ abstracts the notion of independence from linear algebra, graph theory, and combinatorics.
- The three axioms — non-emptiness, heredity, and **exchange** — precisely characterise when greedy is globally optimal.
- All bases of a matroid have equal size, giving a consistent notion of **rank**.
- **Matroid Greedy Theorem (Edmonds 1971):** Greedy finds a maximum-weight basis for every weight function iff the feasible sets form a matroid.
- **Graphic matroids** explain why Kruskal's and Prim's MST algorithms are correct.
- Matroids set the **exact boundary** between greedy algorithms that are provably correct and those that are merely plausible-looking.

Further Reading

References

Reference	Focus
Oxley, J. (2011). *Matroid Theory*, 2nd ed. Oxford University Press.	Comprehensive reference on matroid theory
Edmonds, J. (1971). "Matroids and the Greedy Algorithm." *Math. Programming* 1, 127–136.	Original paper — matroid greedy theorem
Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 4th ed. MIT Press.	MST algorithms, greedy, Union-Find
Schrijver, A. (2003). *Combinatorial Optimization*. Springer.	Matroids, polymatroids, submodularity
Korte, B. & Vygen, J. (2012). *Combinatorial Optimization*, 5th ed. Springer.	Greedoids, matroids, algorithms
Welsh, D. (1976). *Matroid Theory*. Academic Press.	Classic textbook — readable and complete
Lawler, E. (1975). "Matroid Intersection Algorithms." *Math. Programming* 9, 31–56.	Matroid intersection algorithms

Appendix: Equivalent Axiom Systems

A matroid can be equivalently defined via any of the following systems:

System	Key axiom
Independent sets $\mathcal{I}$	(I3) Exchange: smaller $\mathcal{I}$-set can be augmented from larger
Bases $\mathcal{B}$	$\forall B_1, B_2 \in \mathcal{B}$, $\forall e \in B_1 \setminus B_2$, (B2) Basis exchange: $\exists f \in B_2 \setminus B_1 : (B_1 \setminus \{e\}) \cup \{f\} \in \mathcal{B}$
Circuits $\mathcal{C}$	$C_1 \neq C_2$, (C3) Elimination: $e \in C_1 \cap C_2 \Rightarrow \exists C_3 \subseteq (C_1 \cup C_2) \setminus \{e\}$
Rank r	(R3) Submodularity: $r(A \cup B) + r(A \cap B) \leq r(A) + r(B)$
Closure cl	$e \notin \mathrm{cl}(A)$, (CL4) Mac Lane: $e \in \mathrm{cl}(A \cup \{f\}) \Rightarrow f \in \mathrm{cl}(A \cup \{e\})$

- Each system has strengths: circuits for graphic matroids, rank for linear algebra, bases for optimisation.
- Proving all systems equivalent is a rewarding exercise — each equivalence is constructive.